

Postgraduate Lifecycle Management System

Admin Handover Report

Prepared for: Administrator / System Owner
Project: MPhil / PhD Lifecycle Management System
Application URL: <http://localhost:3000> for local development
 : <https://e23-co2060-mplms.vercel.app/> for online testing use
Date: July 8, 2026

Contents

1 Purpose of This Document	1
2 High-Level Project Summary	1
3 Technology Stack	2
4 Architecture Overview	2
4.1 Layered Architecture	2
4.2 Key Source Folders	3
5 Authentication and Access Control	3
5.1 Login Flow	3
5.2 Role Protection	4
6 Database and Storage Design	4
6.1 Core Database Models	4
6.2 Document Storage	4
7 Complete Lifecycle Workflow	5
7.1 Phase 1: Application	5
7.2 Phase 2: Admission	5
7.3 Phase 3: Research Proposal Submission	5
7.4 Phase 4: Supervisor Assignment	6
7.5 Phase 5: Proposal Evaluation	6
7.6 Phase 6: Proposal Final Decision	6
7.7 Phase 7: Ethics Clearance	7
7.8 Phase 8: Progress Reports and Data Collection	7
7.9 Phase 9: Thesis Submission	8
7.10 Phase 10: Examiner Assignment	8
7.11 Phase 11: Viva Scheduling	8
7.12 Phase 12: Viva Outcome	9
7.13 Phase 13: Corrections if Required	9
7.14 Phase 14: Graduation and Final Thesis Archive	10
7.15 Phase 15: Administrative Record Archive	10
8 Workflow Notification Matrix	11
9 CSV Exports and Admin Reports	11
10 Security and Data Integrity	12
11 Administrator Operating Guide	12
11.1 Daily Admin Tasks	12
11.2 Demo Checklist	13
12 Known Implementation Notes	13
13 End-to-End Summary	14

1 Purpose of This Document

This report explains how the Postgraduate Lifecycle Management System is built and how the complete academic workflow runs from application submission to graduation and record archival. It is written for an administrator or system owner who needs to understand:

- what each role does in the system;
- what data is created or updated at each phase;
- who receives email and in-app notifications;
- how the frontend, backend, authentication, database, storage, CSV exports, and reports work together;
- what the administrator should know when operating, demonstrating, or maintaining the project.

Important Operating Principle

The system separates authentication from academic data. Firebase handles login and role claims. Supabase Postgres stores the academic lifecycle records through Prisma. Supabase Storage stores uploaded PDFs using signed URLs.

2 High-Level Project Summary

The project is a role-based web application for postgraduate lifecycle management. It supports four major user groups:

- **Applicant:** submits the public application form before having an account.
- **Student:** submits proposals, ethics approvals, progress reports, theses, and corrections.
- **Supervisor:** reviews proposals, signs off progress reports, and participates in review panels.
- **Examiner:** receives thesis assignments, downloads thesis documents securely, and records viva outcomes.
- **Administrator:** manages admission, users, supervisor/examiner assignments, ethics review, viva scheduling, final thesis archiving, graduation, reports, notification logs, and record archival.

The application follows this broad lifecycle:

**Application → Admission → Proposal → Supervision → Evaluation → Ethics
→ Progress → Thesis → Examination → Viva → Corrections if needed →
Graduation → Archive**

3 Technology Stack

Area	Implementation
Frontend	Next.js 14 App Router, React 18, TypeScript, Tailwind CSS, shadcn-style UI components, Radix primitives, and lucide icons.
Backend	Next.js API routes under <code>src/app/api</code> . Most routes authenticate the request, validate input, call a service in <code>src/lib</code> , then return JSON or CSV.
Authentication	Firebase Authentication for sign-in, Firebase Admin for token/session verification, role claims, and secure session cookies.
Database	Supabase PostgreSQL accessed through Prisma ORM. The schema is defined in <code>prisma/schema.prisma</code> .
File Storage	Supabase Storage using short-lived signed upload and download URLs. Storage roots are separated by workflow.
Email	Nodemailer SMTP sender with templates in <code>src/lib/email.ts</code> . Each delivery attempt is logged.
Notifications	In-app notifications are stored in the <code>Notification</code> table. Delivery audit records are stored in <code>NotificationLog</code> .
Validation	Zod schemas validate request bodies and workflow inputs.
Testing	Vitest and Testing Library cover unit, integration, route, and frontend component behavior.

4 Architecture Overview

4.1 Layered Architecture

Browser / Frontend

Next.js pages, React panels, dashboard layout, forms, tables, modals, notification drawer

↓ HTTP fetch requests and secure cookies

Next.js API Routes

Authentication wrapper, role checks, request parsing, JSON and CSV responses

↓ service calls

Service Layer

Applications, proposals, ethics approvals, progress reports, theses, viva, assignments, archive, reports

↓ Prisma, Firebase Admin, Supabase Storage, SMTP

External / Persistent Services

Firebase Auth, Supabase Postgres, Supabase Storage, SMTP email provider

4.2 Key Source Folders

Path	Purpose
<code>src/app</code>	Next.js pages and route groups.
<code>src/app/api</code>	Backend API endpoints.
<code>src/components</code>	Reusable frontend panels, forms, dashboards, and UI controls.
<code>src/lib</code>	Backend service logic, validation helpers, storage, email, notifications, Firebase, Prisma, reports.
<code>src/lib/firebase</code>	Firebase client/admin integration and authentication middleware.
<code>src/lib/storage.ts</code>	Supabase Storage signed upload and download logic.
<code>src/lib/email.ts</code>	SMTP email sender and email templates.
<code>src/lib/notifications.ts</code>	Central notification dispatcher for email plus in-app notifications.
<code>src/lib/admin</code>	Admin-specific reports, archive, notification log, and monitoring services.
<code>src/lib/ethics</code>	Ethics approval workflow services and schemas.
<code>src/lib/theses</code>	Thesis submission, correction, and versioning services.
<code>prisma/schema.prisma</code>	Database models and enums.
<code>tests</code>	Unit and integration tests.

5 Authentication and Access Control

5.1 Login Flow

1. User enters email and password in the login screen.
2. The frontend signs in using Firebase Authentication.
3. Firebase returns an ID token with a role claim.
4. The frontend sends the ID token to `/api/auth/session`.
5. The backend verifies the token with Firebase Admin.
6. The backend checks the matching Prisma `User` record and confirms that the account is active.
7. A secure HTTP-only session cookie is created.
8. The user is redirected to the dashboard for their role.

5.2 Role Protection

Backend routes use a shared authentication wrapper. The wrapper verifies either a bearer token or a session cookie, resolves the current user, checks the allowed roles, and gives the route an authenticated user context.

Examples:

- Student-only routes: proposal submission, ethics submission, progress report submission, thesis submission, correction upload.
- Supervisor-only routes: proposal evaluation, progress report sign-off.
- Examiner-only routes: assigned viva workspace and viva outcome recording.
- Administrator-only routes: admission, user management, ethics decision, examiner assignment, viva scheduling, final archive, reports, notification log.

6 Database and Storage Design

6.1 Core Database Models

The Prisma schema contains the full academic lifecycle. Important models include:

- | | |
|--------------------|----------------------------|
| • User | • ProgressReport |
| • Student | • ReviewPanel |
| • Supervisor | • Thesis |
| • Examiner | • ThesisExaminerAssignment |
| • Administrator | • Viva |
| • Application | • CorrectionDocument |
| • Registration | • Document |
| • ResearchProposal | • Notification |
| • EvaluationForm | • NotificationLog |
| • EthicsApproval | |

6.2 Document Storage

PDF files are stored in Supabase Storage. The database stores metadata in the **Document** table, including file name, storage path, document type, version, owner links, and soft-delete status.

Protected storage roots:

- applications
- proposals
- ethics-approvals
- theses
- progress-reports
- corrections

The application generates signed upload URLs and signed download URLs. This means files are not publicly exposed by default. Download links expire after a short time.

7 Complete Lifecycle Workflow

7.1 Phase 1: Application

Item	Details
Actor	Applicant.
Page	Homepage, then public application form.
Submitted	Name, email, phone, program type, research area, statement of purpose, and one PDF attachment.
How	The frontend requests a signed Supabase upload URL, uploads the PDF, then submits the application metadata.
System result	Creates Application with status SUBMITTED . Creates Document row for the PDF. No account is created yet.
Notifications	Active administrators receive a new application email. The email attempt is written to NotificationLog .

7.2 Phase 2: Admission

Item	Details
Actor	Administrator.
Page	Admin dashboard, Applications.
Action	Reviews the submitted application and admits or rejects it.
System result for admission	Updates application to ADMITTED , creates a Firebase Auth user, creates Prisma User with role STUDENT , creates Student , and creates initial active Registration .
Notifications	Student receives a welcome email with login URL and temporary password. The email attempt is logged.

7.3 Phase 3: Research Proposal Submission

Item	Details
Actor	Student.
Page	Student dashboard, Proposals.
Submitted	Proposal title, abstract, and proposal PDF.
How	The file is stored in Supabase Storage with a signed upload URL. Metadata is recorded in Prisma.
System result	Creates ResearchProposal , proposal Document , and status SUBMITTED . The progress tracker shows proposal submission as the active stage.
Notifications	Proposal status notifications are sent when status changes later in the workflow.

7.4 Phase 4: Supervisor Assignment

Item	Details
Actor	Administrator.
Page	Admin dashboard, supervisor assignment area.
Action	Assigns at least one supervisor, normally marking one as primary.
System result	Creates SupervisorAssignment . The proposal moves into the review process, normally UNDER_REVIEW .
Notifications	Assigned supervisor receives an email. The delivery attempt is logged.

7.5 Phase 5: Proposal Evaluation

Item	Details
Actor	Supervisor.
Page	Supervisor dashboard, Review Proposals or Proposal Evaluation.
Submitted	Numerical score and academic feedback.
System result	Creates EvaluationForm . Admin can review submitted feedback and score.
Notifications	Administrators receive an email that a proposal evaluation has been submitted. The email is logged.

7.6 Phase 6: Proposal Final Decision

Item	Details
Actor	Administrator.
Page	Admin dashboard, Approve Proposals.
Action	Reviews supervisor feedback and approves or rejects the proposal.
System result	Updates proposal status to APPROVED or REJECTED . If approved, the progress tracker marks proposal approval as complete.
Notifications	Student receives proposal status email and in-app notification. Delivery is logged.

7.7 Phase 7: Ethics Clearance

Item	Details
Student action	Student opens Ethics Approval, enters title and summary, uploads ethics approval PDF, and submits.
Preconditions	Student must have an active registration and an approved proposal.
System result	Creates <code>EthicsApproval</code> with status <code>SUBMITTED</code> . Creates linked ethics <code>Document</code> .
Admin action	Administrator reviews the ethics application and approves or rejects it.
Decision result	Updates status to <code>APPROVED</code> or <code>REJECTED</code> ; stores review notes, reviewer, and review time.
Notifications	Administrators receive email and in-app notification when submitted. Student receives email and in-app notification when status changes.

7.8 Phase 8: Progress Reports and Data Collection

Item	Details
Actor	Student.
Page	Student dashboard, Progress Reports.
Submitted	Period label, narrative, and optional progress report PDF.
System result	Creates <code>ProgressReport</code> . If a PDF is supplied, creates a <code>Document</code> .
Notifications	Primary supervisor receives progress report submitted email and in-app notification.
Actor	Supervisor.
Page	Supervisor dashboard, Sign Progress Reports.
Action	Reviews and signs off the report.
System result	Marks the report signed off, stores sign-off time and supervisor, clears overdue state. If a review panel is assigned, creates panel evaluation tasks.
Notifications	Review panel members receive email when a signed-off report is forwarded to panel review.

7.9 Phase 9: Thesis Submission

Item	Details
Actor	Student.
Page	Student dashboard, Thesis Submission.
Submitted	Thesis title, abstract, and thesis PDF.
Preconditions	Student must be active, have an active registration, and have an approved proposal.
System result	Creates Thesis with status SUBMITTED . Creates current thesis Document . Versioning is supported if a corrected thesis is later submitted.
Notifications	Active administrators receive thesis submitted email. The delivery attempt is logged.

7.10 Phase 10: Examiner Assignment

Item	Details
Actor	Administrator.
Page	Admin dashboard, Examiner Assignments.
Action	Selects thesis and examiner, then adds assignment.
System result	Creates ThesisExaminerAssignment . Generates secure signed thesis download URL for the examiner.
Next admin action	Clicks Start Examination to change thesis status from SUBMITTED to UNDER_EXAMINATION .
Notifications	Examiner receives assignment email with secure thesis download link. The email attempt is logged.

7.11 Phase 11: Viva Scheduling

Item	Details
Actor	Administrator.
Page	Admin dashboard, Schedule Vivas.
Submitted	Thesis, venue, scheduled date and time.
Precondition	Thesis must be UNDER_EXAMINATION .
System result	Creates or updates Viva .
Notifications	Student and assigned examiners receive viva scheduled email and in-app notifications.

7.12 Phase 12: Viva Outcome

Item	Details
Actor	Examiner.
Page	Examiner dashboard, Assigned Vivas.
Action	Downloads thesis through signed URL and records final outcome.
Possible outcomes	PASS, MINOR_CORRECTIONS, MAJOR_CORRECTIONS, FAIL.
System result	PASS moves thesis to FINAL_ARCHIVE. Minor or major corrections move thesis to CORRECTIONS_REQUIRED. Fail moves thesis to CLOSED.
Notifications	The outcome updates thesis state. A separate outcome email is not currently sent at this exact step.

7.13 Phase 13: Corrections if Required

Item	Details
Actor	Student.
Page	Student dashboard, Corrections.
Submitted	Correction type, optional description, and correction PDF.
Precondition	Thesis status must be CORRECTIONS_REQUIRED.
System result	Creates <code>CorrectionDocument</code> and linked correction <code>Document</code> .
Notifications	Administrators receive correction submitted email and in-app notification.
Actor	Administrator.
Page	Admin dashboard, Finalize Theses.
Action	Reviews submitted correction and clicks Approve Correction.
System result	Marks correction approved and records approval time and approving administrator.

7.14 Phase 14: Graduation and Final Thesis Archive

Item	Details
Actor	Administrator.
Page	Admin dashboard, Finalize Theses.
Action	Clicks Archive and Graduate.
Allowed when	Thesis is FINAL_ARCHIVE , usually after PASS , or thesis is CORRECTIONS_REQUIRED with at least one approved correction.
System result	Moves thesis to final archive if needed and updates student academic status to GRADUATED .
Notifications	Student receives thesis archived/graduation email and in-app notification.

7.15 Phase 15: Administrative Record Archive

Item	Details
Actor	Administrator.
Purpose	Remove completed or inactive student records from active operational lists while keeping audit/history data.
System result	Marks student archived and sets academic status to ARCHIVED . Archives linked application, progress reports, research proposals, and ethics approvals.
Warnings	If active thesis pipeline items exist, the archive service returns warnings.
Notifications	Administrative record archive does not currently send a separate email.

8 Workflow Notification Matrix

Event	Recipient	Channel
New application submitted	Administrators	Email, delivery logged.
Application admitted	Student	Welcome email with login details, delivery logged.
Supervisor assigned	Supervisor	Email, delivery logged.
Proposal evaluated	Administrators	Email, delivery logged.
Proposal status changed	Student	Email and in-app notification.
Ethics approval submitted	Administrators	Email and in-app notification.
Ethics status changed	Student	Email and in-app notification.
Progress report submitted	Primary supervisor	Email and in-app notification.
Progress report signed off and forwarded	Review panel members	Email, delivery logged.
Thesis submitted	Administrators	Email, delivery logged.
Examiner assigned	Examiner	Email with secure thesis download link, delivery logged.
Viva scheduled	Student and assigned examiners	Email and in-app notification.
Correction submitted	Administrators	Email and in-app notification.
Thesis archived / graduated	Student	Email and in-app notification.

9 CSV Exports and Admin Reports

The system supports server-side CSV export for admin audit and reporting. Routes accept `format=csv`, build escaped CSV output, and return it with `text/csv` headers.

Export	Description
Notification Log	Admin can filter by recipient, event, status, date range, then export the current page as <code>notification-log.csv</code> .
Student Report	Exports student records, program type, academic status, enrollment date, supervisor, and thesis status.
Thesis Pipeline	Exports thesis records and their current statuses.
Graduation Report	Exports graduated students and archived thesis information.
Overdue Progress Reports	Exports overdue report rows with student, period, days overdue, and supervisor.

10 Security and Data Integrity

- **Role-based authorization:** Routes are restricted to allowed roles before service logic runs.
- **Firestore role claims:** User role is stored in Firestore custom claims and mirrored in Prisma.
- **Active-account check:** Inactive user accounts cannot create sessions.
- **HTTP-only cookies:** Server sessions use secure HTTP-only cookies.
- **Signed storage URLs:** Uploaded and downloaded PDFs use temporary signed URLs.
- **File validation:** Uploads enforce PDF content type and size limits.
- **Soft archival:** Operational records can be archived instead of physically deleted from lifecycle history.
- **Notification audit:** Email delivery attempts are recorded in `NotificationLog`.
- **Workflow status validation:** Application and thesis transitions are checked before updates are accepted.

11 Administrator Operating Guide

11.1 Daily Admin Tasks

- Review newly submitted applications.
- Admit or reject applications.
- Create and manage users.
- Assign supervisors to admitted students.
- Review final proposal decisions.
- Review ethics approval submissions.
- Monitor overdue progress reports.
- Assign examiners after thesis submission.
- Schedule vivas for theses under examination.
- Finalize theses after pass or approved corrections.
- Export reports and notification logs for audit.

11.2 Demo Checklist

Before demonstrating the project:

1. Start the application at `http://localhost:3000`.
2. Confirm Firebase, Supabase, SMTP, and database environment variables are configured in `.env`.
3. Confirm Prisma migrations have been applied to the connected Supabase database.
4. Prepare test accounts for administrator, supervisor, examiner, and student.
5. Use PDF files for application, proposal, ethics, progress, thesis, and correction uploads.
6. If SMTP is not configured, explain that email delivery will fail safely and be logged as `FAILED`.
7. Use the Notification Log page to show auditability.

12 Known Implementation Notes

- Applicant accounts are not created at initial application submission. They are created only when the administrator admits the application.
- Supabase is used for both database and file storage. Firebase is used for authentication.
- The admin dashboard is the visual reference layout for other dashboards.
- Some legacy notification paths are email-only, while newer workflow paths use the central notification service for both email and in-app notifications.
- Record archival hides records from active operational views but preserves historical data for reporting and audit.
- The project has automated tests for core workflows, routes, services, and UI panels.

13 End-to-End Summary

The project implements a complete postgraduate lifecycle:

1. Applicant submits application and PDF.
2. Administrator admits applicant.
3. System creates Firebase user, Prisma user, student profile, and registration.
4. Student submits research proposal.
5. Administrator assigns supervisor.
6. Supervisor evaluates proposal.
7. Administrator approves proposal.
8. Student submits ethics approval.
9. Administrator approves ethics.
10. Student submits progress reports.
11. Supervisor signs off progress.
12. Student submits thesis.
13. Administrator assigns examiner and starts examination.
14. Administrator schedules viva.
15. Examiner records viva outcome.
16. Student submits corrections if required.
17. Administrator approves corrections if required.
18. Administrator archives thesis and graduates student.
19. Administrator archives the complete student record when it should leave active workflow lists.

Final Takeaway

The system is not only a set of forms. It is a controlled lifecycle engine: each phase changes database state, stores documents securely, validates role permissions, sends notifications, logs delivery attempts, and updates the progress view seen by students and administrators.